@demotoprod · BUDGET

Runaway-agent checklist

One corrupted null value broke a ledger agent's formatting tool. It tried to brute-force a fix: more than 15,000 reasoning calls in under an hour, about $50,000 of cloud spend, and database locks that halted live transactions. Nothing capped the calls, the cost or the failures, and every retry re-sent the whole history as billable input. Five controls.

1 A daily budget per agent identity

The incident report's first control: cost-cap thresholds at the service-ID and project level. One key per agent, one budget per key; when it trips the agent stops, not the finance team.

- Give every agent its own API key or service identity; never a shared team key.
- Set a hard daily spend per identity at the provider or gateway, and a soft alert at half.
- A tripped budget stops the agent and pages a person; it never raises itself.

```
agent: ledger-reconcile   identity: svc-ledger-agent   # its own key, nobody else's
daily_budget_usd: 50      # hard stop at the gateway; soft alert at 25
```

2 A circuit breaker on consecutive failures

The loop was the agent retrying its own fix. The report asks for automated financial circuit breakers that halt an agent after a set number of consecutive task failures.

- Count failures per task, not per session; three in a row ends the run and files the task for a human.
- Treat a tool error, a schema failure and an empty result as failures; a retry is not progress.
- Log the last failing input with the task so the fix starts from evidence.

```
max_consecutive_failures: 3   on_trip: halt_task + notify_owner
```

3 A step limit and a depth limit per task

Bounded recursion is the third control in the report. A task that needs more than the limit needs a person, and the limit is what turns a runaway loop into a ticket.

- Cap tool calls per task (25 is a sane default for a reconciliation step) and sub-task depth (2).
- Make hitting the limit a normal, logged outcome, never an exception that triggers a retry.

```
max_steps: 25   max_depth: 2
```

4 Stop re-billing the history

Tool output re-entering the context is billed again on every call. A September 2026 study measured a session's cumulative input at 14,293 times its first call; raw history adds 21 to 36 percent to its cost.

- Summarise tool output before it re-enters the context; keep the raw result in a store read by reference.
- Put a ceiling on context growth per task and on prompt size per call; measure input tokens per call, not only per session.

5 Watch the cost curve, not the total

A loop shows up as calls per minute and input tokens per call climbing together, long before the invoice; the report's observability control would have caught this in minutes.

- Alert when calls per minute per identity and input tokens per call rise together for a few calls.
- Rate-limit each identity at the gateway so a loop degrades instead of spiking.
- Rehearse it: break a tool in staging and confirm the breaker, the budget and the alert fire.

Sources: a September 2026 threat-intelligence special report, case study 6; arXiv 2609.28585, Persistent Billable State. Illustrative values; set yours from your own traffic. @demotoprod

@demotoprod